

WEEKLY ENGINEERING PROGRESS REPORT

Node: Spatial Telemetry Engine (`spatial-telemetry-engine`)

Engineer Details:

Name: Aarav Yadav

Role: AI/ML Software Engineer Intern

Focus: Spatial Data & Computer Vision

1. Accomplishments This Week (Day-by-Day Breakdown)

This week, the project evolved step-by-step from a terminal text search program into a full-stack document assistant, before completely pivoting into a specialized data engine for 3D driving logs and vehicle coordinates.

Monday (Terminal-Based Text Search Setup)

- Set up a simple text search program (RAG) that runs directly inside the command line terminal.
- Built a connection to local AI model tools using Ollama (`nommic-embed-text` and `llama3.2`) to read text blocks without needing internet cloud APIs.
- Wrote basic logic to break text into smaller parts so the database could analyze simple sentences.

Tuesday (Full-Stack Text Search Expansion)

- Upgraded the project from a terminal script into a multi-layer full-stack application.
- Added support for PDF document uploads and raw text inputs to easily ingest massive data.
- Built a working backend connection to a local database (PostgreSQL + `pgvector`) to safely store text segments alongside their mathematical AI representations (embeddings) via an incoming FastAPI gateway.

Wednesday (Architecture Diagrams & Documentation)

- Documented the entire baseline system architecture and workflow maps inside the repository's README.md file.
- Designed comprehensive architecture, flow, and structural sequence diagrams detailing exactly how user queries step through data blocks.
- Added strict text file path tracing rules into the structural diagram layout to map exactly how files are digested by the server.

Thursday (The Spatial Pivot & Matrix Ingestion)

- Completely shifted the project focus away from standard text documents to support physical driving telemetry.
- Rebuilt the database rules to accept 3D bounding boxes, camera sensor orientations, and car velocities.
- Created strict structural code formats (`BoundingBox3D` and `SpatialFramePayload`) to guarantee vehicle positions always arrive in clean 3D formats ($[X, Y, Z]$), calculating the true speed of the car ($\|v\| = \sqrt{V_x^2 + V_y^2 + V_z^2}$) to turn raw numbers into clear physical descriptions.

- Set up the main query system to find past driving scenes in under 5 milliseconds, outputting a standard 4D state vector initialization layout ($x = [X, Y, \dot{X}, \dot{Y}]^T$) to kick off downstream vehicle tracking filters.

Friday – Today (User Interface & Simulation Deployment)

- Developed a clean, interactive user dashboard (`ui.py`) using Streamlit, giving users a workspace to visually paste driving log files or type out traffic scenario searches.
- Programmed a mock data simulation script (`test_ingest.py`) to mimic a front-end vehicle camera sensor. This automatically generates fake traffic matrices and passes them directly to our endpoints to test database loading speeds.

2. Next Week's Targets

- **Complete Real-World Dataset Bridge & Speed Testing**

Write a data converter utility to read raw vehicle logs from massive open autonomous driving datasets (like nuScenes/Waymo). This will feed real tracking matrices into our local platform to stress-test and optimize search lookup speeds under high enterprise database stress.

Report compiled securely on Arch Linux Host PC Maomao. Internal metrics verified.